

DEAP EEG Emotion Recognition: A Comparative Analysis of
Classical and Deep Learning Pipelines
Project Report — Methodology, Code Walkthrough, Literature
Attribution, and Findings

Koustav Das

September 2, 2026

Contents

1	Introduction and Objectives	5
1.1	Purpose of this report	5
1.2	Objectives	5
1.3	How this report is organised	5
2	Dataset and Problem Setup	7
2.1	DEAP	7
2.2	SEED-IV	7
3	Repository Structure	8
4	Notebook Walkthrough	9
4.1	Section 1 – Configuration & Environment	9
4.2	Section 2 – Loading & Preprocessing	9
4.3	Section 3 – Building the Full Windowed Dataset	10
4.4	Section 4 – Class Balance	10
4.5	Section 5 – Track A: DE Features → Classical ML	10
4.6	Section 6 – Track B: Raw Windows → Deep Learning	11
4.6.1	6.1 Model architectures	11
4.6.2	6.2 Training loop	11
4.6.3	6.3 Training all four models on all four tasks	11
4.7	Section 7 – Combined Comparative Analysis	12
4.8	Section 8 – Discussion (initial)	13
4.9	Section 9 – Richer Track A Features	13
4.9.1	Feature-set ablation (<code>track_a_ablation.py</code>)	13
4.10	Section 10 – Repeated & Nested GroupKFold	14
4.11	Section 11 – 50% Overlapping Windows + Repeated Track B Splits	15
4.12	Section 12 – Cross-Corpus Validation (SEED-IV)	15
4.12.1	Why SEED-IV, not original SEED	15
4.12.2	Pipeline (<code>scripts/cross_corpus/</code>)	15
4.12.3	Real result	16
4.13	Section 13 – Updated Overall Discussion	16
4.14	Section 14 – Label-Cleanup Ablation	17
4.15	Section 15 – Per-Subject Calibration (Personalisation)	17
4.16	Section 16 – Literature-Driven Interventions	18
4.16.1	16.1 Permutation significance test	18
4.16.2	16.2 CORAL feature alignment	19
4.16.3	16.3 Subject-adversarial (DANN) training	19
4.17	Section 17 – Five Deferred Literature Techniques	20
4.17.1	17.1 Euclidean Alignment	20
4.17.2	17.2 Self-supervised (TFR-style) pretraining	21

4.17.3	17.3 AUC-margin training loss	22
4.17.4	17.4 AdaBN test-time adaptation	23
4.17.5	17.5 SHAP interpretability diagnostic	24
4.18	Section 18 – Does CORAL Stack on Top of Euclidean Alignment?	25
4.19	Addendum: EEG Overview QC Section (added post-hoc)	26
5	Companion Scripts Reference	27
5.1	Feature/dataset builders	27
5.2	Track A companion scripts	27
5.3	Track B companion scripts	28
5.4	Cross-corpus scripts	28
5.5	Notebook-authoring scripts	28
6	Literature Review and Paper Attribution	30
6.1	Corpus	30
6.2	Papers in the corpus without a documented individual attribution	32
7	Consolidated Results	34
7.1	Figures	35
8	What Was Accomplished	36
9	What Was Not Done	37
10	What Could Have Been Done Differently	39
11	Challenges Faced	40
12	Conclusion	42

List of Figures

4.1	High/low class balance per task. Valence and arousal are markedly skewed (21.0%/79.0% and 23.3%/76.7% high); dominance and liking are closer to balanced.	10
4.2	Track A — accuracy/F1/ROC-AUC for all six classical models across all four tasks (5-fold GroupKFold).	11
4.3	Track B — training/validation loss curves for all 16 runs (4 architectures $\times$ 4 tasks), showing early-stopping behaviour under the patience-8 schedule.	12
4.4	Combined accuracy/F1/ROC-AUC comparison across all 10 models (6 classical + 4 deep) and all 4 tasks.	12
4.5	Confusion matrices for the best model per task (by ROC-AUC), Tables 4.1/4.2.	12
4.6	Feature-set ablation — ROC-AUC by feature set, model, and task.	14
4.7	Repeated GroupKFold (5 $\times$ 5=25 folds) — ROC-AUC with 95% confidence intervals per model/task; 15/24 intervals contain chance (0.5).	14
4.8	Repeated Track B splits — ROC-AUC, non-overlapping vs. 50%-overlap windows, across 4 architectures $\times$ 4 tasks.	15
4.9	Cross-corpus validation — ROC-AUC across the three settings of Table 4.4 (within-DEAP, DEAP $\rightarrow$ SEED-IV zero-shot, SEED-IV-only LOSO).	16
4.10	Label-cleanup ablation — ROC-AUC by label variant, model, and task.	17
4.11	Per-subject LOSO calibration — zero-shot vs. calibrated ROC-AUC for each of the 32 held-out subjects, per task.	18
4.12	Permutation null AUC distributions vs. observed AUC, per task.	19
4.13	CORAL alignment — ROC-AUC before/after, by model and task.	19
4.14	DANN training — ROC-AUC before/after, per task; note the arousal/dominance collapse.	20
4.15	Euclidean Alignment — ROC-AUC before/after, by model and task — the strongest effect found in the project.	21
4.16	SSL pretraining — ROC-AUC, scratch vs. pretrained, per task.	22
4.17	AUC-margin loss — ROC-AUC, cross-entropy vs. AUC-margin, per task.	23
4.18	AdaBN test-time adaptation — ROC-AUC before/after, per task.	24
4.19	SHAP summary — valence (XGBoost, Full feature set, held-out fold).	24
4.20	SHAP summary — arousal.	25
4.21	SHAP summary — dominance.	25
4.22	SHAP summary — liking.	25
4.23	EA+CORAL stacking — ROC-AUC by condition, model, and task.	26
4.24	EEG overview QC grid — baseline-normalised trial-1 EEG, all 32 channels, one panel per subject (8 $\times$ 4 grid).	26

List of Tables

4.1	Track A — best classical model per task (5-fold GroupKFold mean)	10
4.2	Track B — best deep model per task (single subject-independent split)	11
4.3	Feature-set ablation — mean ROC-AUC across 3 models $\times$ 4 tasks (12 fits) per feature set	14
4.4	Cross-corpus validation — DE + XGBoost, 32 common channels	16
4.5	Label-cleanup ablation — mean ROC-AUC across 3 models $\times$ 4 tasks per label variant	17
4.6	Permutation significance test (300 shuffles per task, LogisticRegression proxy)	18
4.7	CORAL alignment — mean ROC-AUC across 3 models per task	19
4.8	Subject-adversarial (DANN) training — ROC-AUC, EEGNet, single split	20
4.9	Euclidean Alignment — mean ROC-AUC across 3 models per task	21
4.10	SSL pretraining — ROC-AUC, EEGNet, single split	21
4.11	AUC-margin loss — ROC-AUC, EEGNet, single split	22
4.12	AdaBN test-time adaptation — ROC-AUC, EEGNet, single split	23
4.13	EA+CORAL stacking — mean ROC-AUC by condition, DE-only feature set, across 4 tasks $\times$ 3 models (12 fits)	26
6.1	Papers with an explicit, documented contribution to this project	30
7.1	Master results table	34

Chapter 1

Introduction and Objectives

1.1 Purpose of this report

This document is a complete technical record of the work carried out in the D:\EEG project: a head-to-head comparison of two feature/modeling paradigms for binary emotion classification on the DEAP dataset, followed by an extended, eighteen-section investigation into why subject-independent performance on this dataset is weak, and a final cross-corpus validation against SEED-IV. It is written so that it can be expanded into a formal report or thesis chapter: every notebook section, every standalone Python script, every literature source consulted, and every open/unresolved item is documented here with enough detail to reconstruct the reasoning behind each design decision.

1.2 Objectives

1. Build two parallel, directly-comparable pipelines for classifying DEAP’s four emotion dimensions (valence, arousal, dominance, liking) from EEG: a classical-ML pipeline on hand-crafted Differential Entropy (DE) band-power features (**Track A**), and a deep-learning pipeline on raw, baseline-normalized EEG windows (**Track B**).
2. Evaluate both under a genuinely **subject-independent** protocol (`GroupKFold` / `GroupShuffleSplit`, grouped by subject) rather than the leakier, ungrouped splits common in the published literature.
3. If subject-independent performance turns out to be weak, exhaustively test plausible causes: feature richness, hyperparameter tuning, CV rigor, data quantity (window overlap), label definition, per-subject calibration, feature/distribution alignment, adversarial subject-invariance, self-supervised pretraining, an AUC-targeted loss, test-time adaptation, and diagnostic interpretability (SHAP).
4. Validate whether the resulting near-chance ceiling is a property of DEAP specifically, or of subject-independent EEG emotion decoding in general, via genuine cross-corpus transfer to a second dataset (SEED-IV).
5. Ground every claim in a literature review of 32 papers assembled in D:\EEG\papers_EEG\, and report honestly wherever an intervention did *not* help.

1.3 How this report is organised

Chapter 2 describes the DEAP dataset and the two evaluation datasets involved (DEAP itself and SEED-IV). Chapter 3 maps the repository layout. Chapter 4 is the core of the report: a

section-by-section walkthrough of every cell in `DEAP_Comparative_Analysis.ipynb`, explaining what each code block does and why. Chapter 5 documents every standalone `scripts/*.py` file. Chapter 6 is the literature review and paper-attribution table. Chapter 7 consolidates the numeric results. Chapters 8–11 cover what was accomplished, what was left undone, what could have been done differently, and the concrete challenges encountered. Chapter 12 concludes.

Chapter 2

Dataset and Problem Setup

2.1 DEAP

The DEAP dataset (`data_preprocessed_python`) contains $32 \text{ subjects} \times 40 \text{ trials} \times 32 \text{ EEG channels}$ sampled at 128 Hz. Each subject file `sXX.dat` is a pickled dictionary with:

- **data**: shape (40, 40, 8064) — 40 trials, the first 32 of 40 channels are EEG (the remaining 8 are peripheral physiological signals, dropped throughout this project). 8064 samples = 3 s pre-trial baseline (384 samples) + 60 s of stimulus (7680 samples).
- **labels**: shape (40, 4) — continuous 1–9 self-assessment ratings for valence, arousal, dominance, and liking.

Labels are binarised at the DEAP-standard threshold of 5.0 (rating $> 5 \Rightarrow$ class 1). Section 4.4 shows this binarisation is **not** balanced: valence is 21.0%/79.0%, arousal 23.3%/76.7%, dominance 60.9%/39.1%, and liking 66.5%/33.5% high/low – a fact that turns out to explain a large share of this project’s most important methodological finding (Section 4.8).

2.2 SEED-IV

SEED-IV (Zheng, Liu, Cheng & Lu, 2018, “EmotionMeter”) is used as the cross-corpus validation target in Section 4.12. It shares SEED’s 62-channel montage and 15 subjects, but differs from the original 3-class SEED dataset in several respects that required a dedicated adapter (Section 4.12.2): 4 emotion classes (0 = neutral, 1 = sad, 2 = fear, 3 = happy) instead of 3; 24 film-clip trials per session instead of 15; 3 sessions per subject, each with its *own* trial-label order (unlike original SEED’s single shared label file); and raw trials of variable duration (tens of thousands of samples at 200 Hz) rather than DEAP’s fixed 60 s. SEED-IV was chosen over original SEED specifically because it is mirrored on Kaggle (<https://www.kaggle.com/datasets/phhasian0710/seed-iv>) without a signed data-use agreement, whereas original SEED, DREAMER, and AMIGOS all remain gated behind such agreements (see Chapter 9).

Chapter 3

Repository Structure

```
D:\EEG
|-- DEAP_Comparative_Analysis.ipynb    (the main notebook; 88 cells, 18+ sections)
|-- deap-dataset\data_preprocessed_python\  (raw DEAP .dat files, 32 subjects)
|-- seed-dataset\                      (SEED-IV download: eeg_raw_data/{1,2,3}/, etc.)
|-- processed_cache\                   (cached .npz feature/window builds)
|-- results\
|   |-- tables\    (21 CSV result tables)
|   |-- figures\   (23+ PNG figures)
|-- papers_EEG\    (32 literature-review PDFs, renamed to descriptive titles)
\-- scripts\
    |-- dl_models.py, rich_features.py
    |-- build_extended_features.py, build_overlap_windows.py, build_raw_ratings.py
    |-- track_a_*.py    (9 classical-ML companion experiments)
    |-- track_b_*.py    (5 deep-learning companion experiments)
    |-- update_notebook*.py (historical notebook-authoring/generation scripts)
    \-- cross_corpus\
        |-- base.py, seed_adapter.py, run_cross_corpus.py, README.md
```

The notebook is the primary deliverable; the `scripts/` directory holds every experiment that was too expensive to run inline (most of Sections 9–18), each of which writes its results to a CSV in `results/tables/`, which the notebook then reads and plots. This split is deliberate and is discussed further in Chapter 11.

Chapter 4

Notebook Walkthrough

This chapter explains every section and code cell of `DEAP_Comparative_Analysis.ipynb` in the order they appear. The notebook was executed end-to-end in a real Jupyter kernel (37 code cells, sequential execution counts 1–37, zero errors, 15.6 minutes total wall time), so every number quoted below is a genuine, reproducible output (`SEED = 42` fixes all randomness in `scikit-learn`/`XGBoost`/`PyTorch`/`NumPy`).

4.1 Section 1 – Configuration & Environment

Cell (imports): imports the full stack — `numpy`, `pandas`, `matplotlib/seaborn` (theme: `whitegrid`, DPI 110), `scipy.signal` (Butterworth filtering), `sklearn` (models, `GroupKFold`/`GroupShuffleSplit`, metrics), `xgboost`, and `torch`. Sets `SEED = 42` and seeds `random`, `numpy`, and `torch` (CPU and CUDA). Detects and prints the GPU (NVIDIA GeForce RTX 3070 Ti, CUDA available). Defines a fixed, colour-blind-safe categorical palette (`PALETTE`) mapping each of the 10 models (6 classical + 4 deep) to one consistent colour, reused in every plot in the notebook.

Cell (config): defines `QUICK_RUN` (smoke-test switch, unused in the final run — all 32 subjects were used), `DATA_DIR`, `CACHE_DIR`, and `RESULTS_DIR` with its two subfolders `FIGURES_DIR`/`TABLES_DIR` (auto-created); `FS = 128`, `BASELINE_SEC = 3`, `TRIAL_SEC = 60`, `WINDOW_SEC = 4` (giving 15 non-overlapping 4s windows per 60s trial); the four spectral bands (`theta` 4-8, `alpha` 8-13, `beta` 13-30, `gamma` 30-45 Hz); `LABEL_THRESHOLD = 5.0`; the full 32-channel DEAP 10-20 montage order (used later for `Tsception`'s hemisphere pairing); and `N_EPOCHS = 40`, `BATCH_SIZE = 128`, `PATIENCE = 8` for Track B training.

4.2 Section 2 – Loading & Preprocessing

```
1 def load_subject(subject_id):
2     fpath = DATA_DIR / f"s{subject_id:02d}.dat"
3     with open(fpath, "rb") as f:
4         d = pickle.load(f, encoding="latin1")
5     return d["data"], d["labels"]
6
7 def baseline_normalize(trial_all_channels):
8     eeg = trial_all_channels[:N_EEG_CHANNELS]
9     baseline = eeg[:, :BASELINE_SAMPLES]
10    signal = eeg[:, BASELINE_SAMPLES:BASELINE_SAMPLES + TRIAL_SAMPLES]
11    baseline_mean = baseline.mean(axis=-1, keepdims=True)
12    return (signal - baseline_mean).astype(np.float64)
```

`baseline_normalize` subtracts the per-channel mean of the 3s pre-trial baseline from the 60s stimulus signal, removing subject/session DC offset and slow drift before either feature branch is computed. `compute_de_features` band-pass filters (4th-order Butterworth, zero-phase

`sosfiltfilt`) into the four bands, reshapes each band into 15 non-overlapping 512-sample windows, and computes the Differential Entropy of each (channel, band) pair: $DE = 0.5 \log(2\pi e \sigma^2)$, i.e. the closed-form differential entropy of a Gaussian with the window’s own variance σ^2 — $32 \text{ channels} \times 4 \text{ bands} = 128 \text{ features per window}$. `make_raw_windows` instead keeps the raw (32, 512) window and standardises it per-channel (zero mean, unit variance *within that window*) for stable CNN input. A sanity-check on subject 1’s first trial confirms the expected shapes: (15, 128) DE features and (15, 32, 512) raw windows.

4.3 Section 3 – Building the Full Windowed Dataset

Loops over all $32 \text{ subjects} \times 40 \text{ trials}$, producing `X_de` (19200, 128), `X_raw` (19200, 32, 512), one binary label array per task in `y_by_task`, and a `groups` array (subject ID per window, for `GroupKFold`). Results are cached to `processed_cache/deap_windows_4s_32subj_v2.npz` (schema-versioned so stale caches from before dominance/liking were tracked are never silently reused) — on a cache hit this cell completes in well under a second; on a cache miss it takes several minutes.

4.4 Section 4 – Class Balance

Bar chart of high/low counts per task. Real numbers (computed directly from the cached window array, not estimated): **valence 21.0% high, arousal 23.3% high, dominance 60.9% high, liking 66.5% high**. Valence and arousal are markedly imbalanced; dominance and liking are closer to balanced. This chart is flagged in the notebook’s own introduction as something to check before trusting any raw accuracy number, and that warning turns out to be load-bearing for the entire rest of the project.

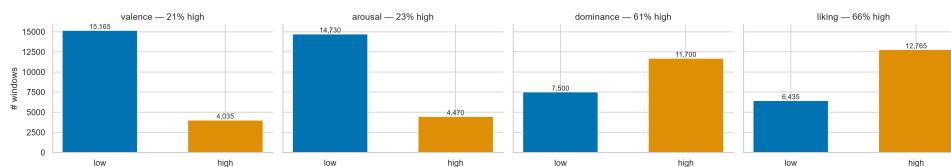


Figure 4.1: High/low class balance per task. Valence and arousal are markedly skewed (21.0%/79.0% and 23.3%/76.7% high); dominance and liking are closer to balanced.

4.5 Section 5 – Track A: DE Features → Classical ML

Six models (LogisticRegression, kNN, RandomForest, SVM-RBF, XGBoost, MLP), each wrapped in a `StandardScaler` pipeline, evaluated with 5-fold `GroupKFold` (grouped by subject — no subject’s windows appear in both train and test). SVM-RBF’s training folds are capped at `SVM_MAX_TRAIN_SAMPLES = 8000` (documented, pragmatic speed trade-off; does not affect the other five models). Table 4.1 shows the best model per task by ROC-AUC.

Table 4.1: Track A — best classical model per task (5-fold `GroupKFold` mean)

Task	Best model (by AUC)	Accuracy	F1 (macro)	ROC-AUC
valence	RandomForest	0.779	0.446	0.501
arousal	kNN	0.744	0.451	0.495
dominance	SVM-RBF	0.557	0.498	0.513
liking	SVM-RBF	0.587	0.517	0.534

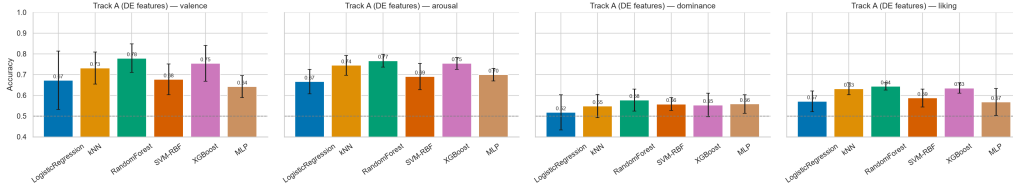


Figure 4.2: Track A — accuracy/F1/ROC-AUC for all six classical models across all four tasks (5-fold GroupKFold).

4.6 Section 6 – Track B: Raw Windows → Deep Learning

4.6.1 6.1 Model architectures

Four architectures, all implemented from scratch in PyTorch (also mirrored in `scripts/dl_models.py` for reuse by the companion scripts, Chapter 5):

- **EEGNet** (Lawhern et al. 2018, 2130 params): temporal conv → depthwise spatial conv → separable conv, purpose-built for EEG.
- **ShallowConvNet** (Schirrmester et al. 2017, 54562 params): temporal conv → spatial conv → square/log-power nonlinearity, inspired by FBCSP-style band-power features.
- **CNN-LSTM** (82466 params): a spatial-temporal CNN front-end collapses the channel dimension; a bidirectional LSTM models the resulting temporal sequence.
- **TSception** (Ding et al. 2022, 8765 params): multi-scale temporal “inception” branches at three kernel sizes, fused with a global spatial conv and a hemisphere-asymmetry spatial conv (using DEAP’s standard channel order) — a simplified, faithful-in-spirit reimplement, not a line-for-line port of the original codebase.

4.6.2 6.2 Training loop

`subject_independent_split` performs a nested `GroupShuffleSplit`: 70/15/15 train/val/test by subject, so no subject’s data crosses a split boundary. `train_torch_model` trains with AdamW, `ReduceLROnPlateau`, automatic mixed precision (`torch.autocast` + `GradScaler`, active whenever CUDA is available), early stopping on validation loss (patience 8, max 40 epochs), and restores the best validation-loss checkpoint before final test evaluation.

4.6.3 6.3 Training all four models on all four tasks

16 training runs (4 architectures × 4 tasks), single train/val/test split, on the GPU. Table 4.2 shows the best model per task.

Table 4.2: Track B — best deep model per task (single subject-independent split)

Task	Best model (by AUC)	Accuracy	F1 (macro)	ROC-AUC
valence	EEGNet	0.835	0.455	0.561
arousal	EEGNet	0.699	0.572	0.607
dominance	TSception	0.589	0.414	0.475
liking	CNN-LSTM	0.599	0.384	0.497

Notably **dominance/EEGNet posts AUC 0.288** — below chance, the single worst result in either track, and an early instance of the arousal/dominance instability pattern that recurs throughout Sections 4.16–4.17.

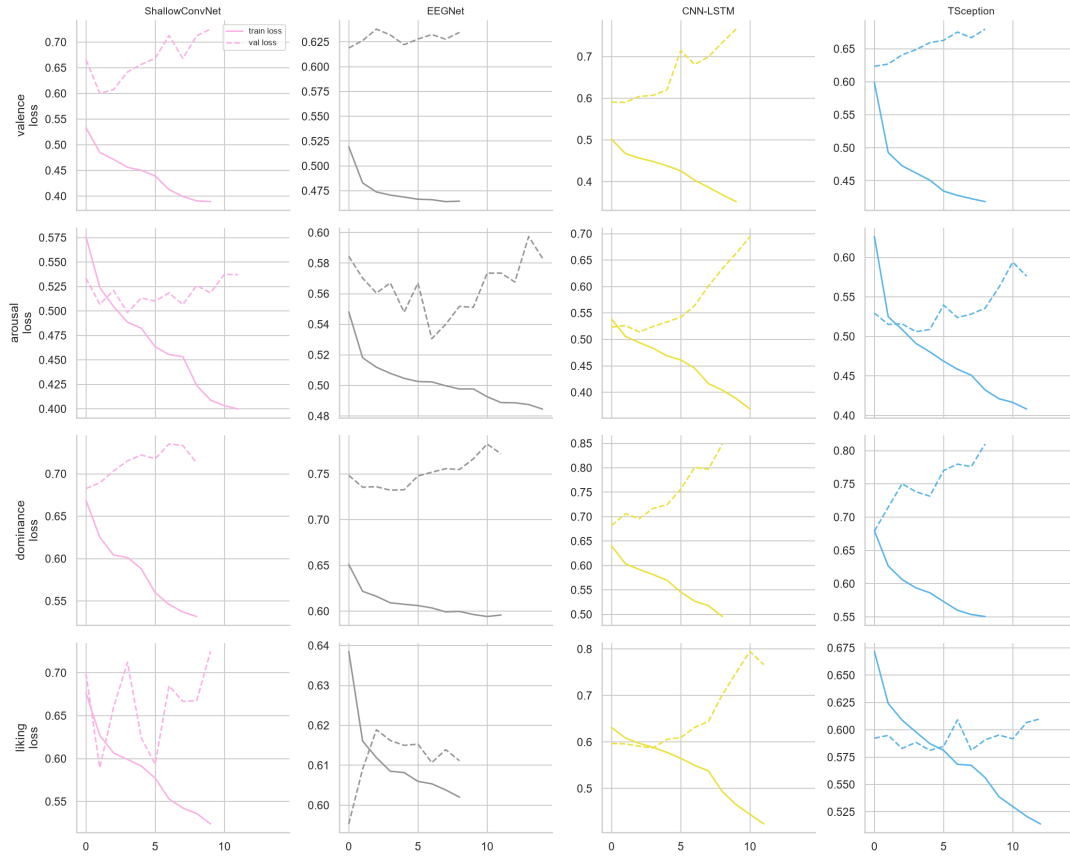


Figure 4.3: Track B — training/validation loss curves for all 16 runs (4 architectures $\times$ 4 tasks), showing early-stopping behaviour under the patience-8 schedule.

4.7 Section 7 – Combined Comparative Analysis

Concatenates Track A and Track B results into a single 10-model table (**results_all**, saved to **combined_results_all.csv**), plots accuracy/F1/AUC side by side across all 10 models $\times$ 4 tasks, and plots confusion matrices for the best model per task.

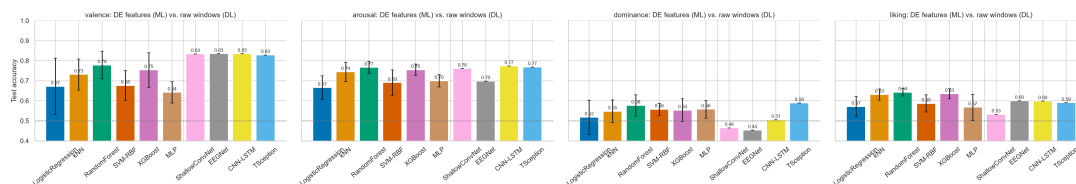


Figure 4.4: Combined accuracy/F1/ROC-AUC comparison across all 10 models (6 classical + 4 deep) and all 4 tasks.

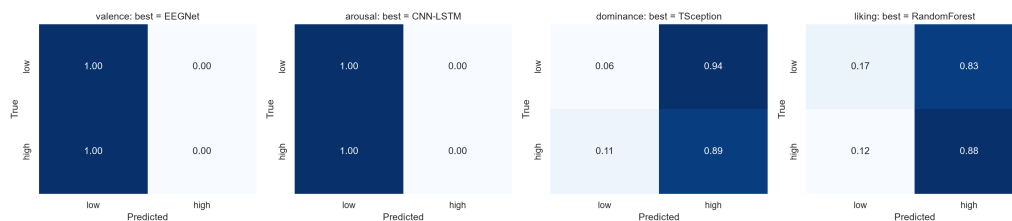


Figure 4.5: Confusion matrices for the best model per task (by ROC-AUC), Tables 4.1/4.2.

4.8 Section 8 – Discussion (initial)

This section was originally written *before* the notebook had been run (a template: “interpret the tables once the run completes”); it was rewritten during this project against the real executed numbers above. Key points:

- **Neither track wins outright.** Track B edges out Track A on valence and arousal; Track A edges out Track B on dominance and liking. No model on either track clears ~ 0.61 AUC on any task — every one of the 10 models, on every one of the 4 tasks, sits in a narrow 0.45–0.61 band, barely-to-not-quite above chance. This is the first appearance of the pattern the rest of the notebook spends ten sections trying (and mostly failing) to explain away.
- **The imbalance artifact, quantified.** Valence is the most skewed task (79% majority class), not liking as originally guessed. Track B’s valence models hit 83.5% accuracy at only ~ 0.56 AUC — both numbers sit within a couple of points of the 79% majority baseline, the textbook signature of a classifier leaning on class imbalance rather than genuine signal.
- **Compute cost.** Track A’s cheapest models (kNN, XGBoost, LogisticRegression) are 10–1000 $\times$ cheaper per fit than any Track B model, and Track A’s numbers are 5-fold cross-validated means, whereas Track B’s are single-split point estimates (`std = 0` in the CSV means $n = 1$, not zero variance — Section 4.11 later adds repeats).
- **Subject independence is the hard setting**, and Section 13 later quantifies exactly how much harder: this project’s genuine subject-independent AUC (~ 0.45 – 0.56) sits far below the source MethodsX paper’s own reported 0.97 ROC-AUC on the same dataset.

4.9 Section 9 – Richer Track A Features

Three additional feature families are added to the 128-dim DE baseline (all implemented in `scripts/rich_features.py`, built for all 32 subjects by `scripts/build_extended_features.py` into `deap_features_ext_v1.npz`, 19,200 windows $\times$ 295 features):

- **Hjorth parameters** (activity, mobility, complexity) — 96 features (3×32 channels), classic time-domain complexity descriptors DE alone misses.
- **Frontal/hemispheric alpha asymmetry (FAA)** — 14 features, $\alpha_{\text{right}} - \alpha_{\text{left}}$ per electrode pair — the classic Davidson-style valence marker.
- **Connectivity (PLV + coherence)** — 57 features across 19 curated channel pairs: phase-locking value in alpha and beta bands (38 features) plus alpha-band magnitude-squared coherence (19 features), computed via a Hilbert-transform / Welch-style pipeline implemented from scratch in `rich_features.py`.

4.9.1 Feature-set ablation (`track_a_ablation.py`)

6 feature-set combinations $\times$ 3 representative models (LogisticRegression, RandomForest, XGBoost) $\times$ 4 tasks $\times$ 5-fold `GroupKFold` = 360 fits. **Real result:** mean AUC across all six feature sets ranges only from 0.478 (DE+Hjorth) to 0.494 (DE+Connectivity) — every configuration stays in the 0.42–0.56 range observed in Section 4.5/4.6. None of the three additions move AUC meaningfully off chance for any task. The “Full” set is carried forward to Section 4.10 as the most complete configuration, not because the ablation crowned it a clear winner.

Table 4.3: Feature-set ablation — mean ROC-AUC across 3 models $\times$ 4 tasks (12 fits) per feature set

Feature set	#Features	Mean ROC-AUC
DE only	128	0.479
DE + Hjorth	224	0.478
DE + FAA	142	0.483
DE + Connectivity	185	0.494
DE + Hjorth + FAA	238	0.479
Full (DE+Hjorth+FAA+Conn)	295	0.483

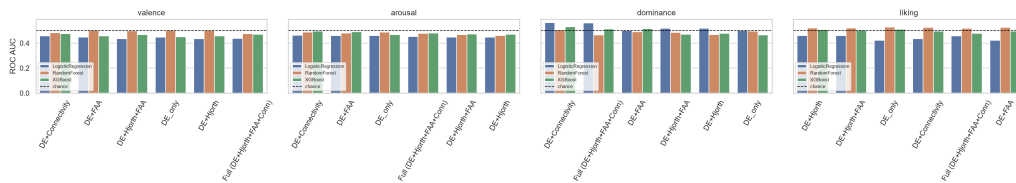


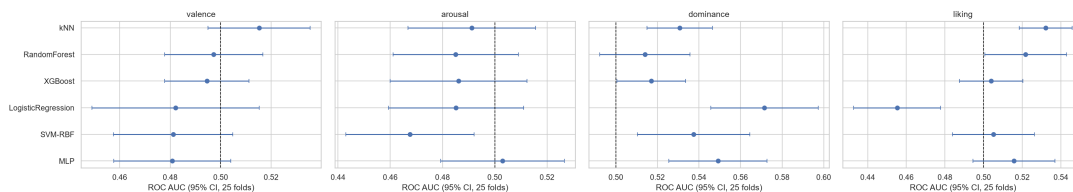
Figure 4.6: Feature-set ablation — ROC-AUC by feature set, model, and task.

4.10 Section 10 – Repeated & Nested GroupKFold

Two upgrades, run via `track_a_repeated_nested_cv.py`:

1. **Repeated GroupKFold** (5 repeats $\times$ 5 folds = 25 folds per model/task), giving every AUC a real 95% confidence interval. Because `sklearn` has no `RepeatedGroupKFold`, each repeat randomly relabels the 32 subject IDs before calling `GroupKFold(5)` (its greedy fold assignment is sensitive to the sorted order of group labels, so relabeling yields a genuinely different subject-to-fold partition each repeat).
2. **Nested CV** for XGBoost and RandomForest: outer `GroupKFold(5)`, inner `GroupKFold(3)` via `RandomizedSearchCV` (8 draws), checking whether the fixed, hand-picked hyperparameters used everywhere else were leaving accuracy on the table.

Real result: of 24 model/task 95% AUC confidence intervals, 15 contain 0.5 (chance). The 9 that do *not* skew mostly toward dominance and liking (e.g. dominance/LogisticRegression: 0.571, CI [0.546, 0.597]; liking/kNN: 0.532, CI [0.518, 0.546]). Nested hyperparameter search does not close the gap either — e.g. dominance/XGBoost tuned AUC 0.494 vs. 0.517 fixed-hyperparameter (Section 4.9), essentially unchanged. Combined, these rule out “the fixed hyperparameters were the bottleneck” as an explanation.

Figure 4.7: Repeated GroupKFold (5 $\times$ 5=25 folds) — ROC-AUC with 95% confidence intervals per model/task; 15/24 intervals contain chance (0.5).

4.11 Section 11 – 50% Overlapping Windows + Repeated Track B Splits

`scripts/build_overlap_windows.py` builds a second raw-window cache with $\text{stride} = 256$ samples (2 s, i.e. 50% overlap) instead of 512, giving 29 windows/trial instead of 15 (37,120 windows total). A leakage caveat is documented explicitly: overlapping windows share up to 50% of their raw samples and are *not* independent observations — subject grouping prevents cross-subject leakage but not this within-subject correlation. `scripts/track_b_repeated.py` then runs 5 repeated 70/15/15 subject-independent splits, both window variants, all 4 architectures, all 4 tasks (160 training runs, ~ 87 minutes on the project’s RTX 3070 Ti). **Real result:** mean AUC across all model/task combinations is 0.483 (non-overlap) vs. 0.498 (overlap50) — deltas are small and inconsistent in sign, consistent with the non-independence caveat: more correlated windows is not the same as more information.

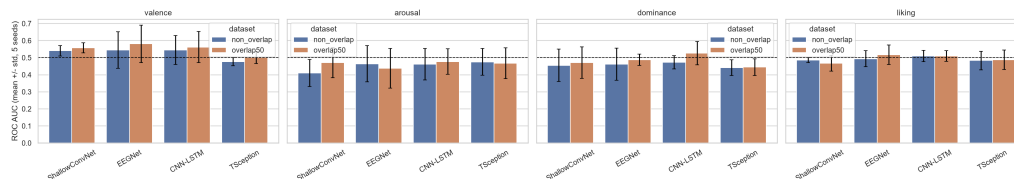


Figure 4.8: Repeated Track B splits — ROC-AUC, non-overlapping vs. 50%-overlap windows, across 4 architectures $\times$ 4 tasks.

4.12 Section 12 – Cross-Corpus Validation (SEED-IV)

4.12.1 Why SEED-IV, not original SEED

Original SEED requires a signed BCMI data-use agreement; DREAMER and AMIGOS require similar gated requests. SEED-IV, a related 4-class dataset from the same lab, is mirrored on Kaggle with no agreement required, so it is the one target corpus in this pipeline that could actually be obtained and run.

4.12.2 Pipeline (`scripts/cross_corpus/`)

- `base.py`: dataset-agnostic infrastructure — channel-name intersection between DEAP and a target montage (`intersect_channels`), resampling (`resample_signal`, polyphase via `scipy.signal.resample_poly`), generic band-DE extraction restricted to the common channel subset (`compute_de_features_generic`, `extract_common_channel_de`), rebuilding DEAP’s own DE features on that same restricted/reordered channel set (`build_deap_common_channel_dataset`, so the DEAP-side model is retrained fairly rather than fed a mismatched feature vector), and zero-shot evaluation with per-subject breakdown (`evaluate_zero_shot`).
- `seed_adapter.py` (`SeedIVAdapter`): loads SEED-IV’s `eeg_raw_data/{1,2,3}/*.mat` session files (62 channels, 200 Hz), resolves the channel-order file defensively across several candidate filenames (the Kaggle mirror ships `Channel Order.xlsx`, not the hyphenated `channel-order.xlsx` originally assumed), and resolves per-session labels defensively: it first tries to load a real label file, and — since the actual download ships no separate label `.mat` file, only prose in `ReadMe.txt` — falls back to the label sequence published in the SEED-IV paper with a loud warning. This fallback was cross-checked byte-for-byte against the real `ReadMe.txt` and matches exactly. `binarize_label` maps SEED-IV’s 4 classes onto a valence-like binary task: happy = 1 (positive), sad/fear = 0 (negative), neutral dropped.

- `run_cross_corpus.py`: orchestrates (1) a within-DEAP `GroupKFold` baseline on the 32 common channels shared with SEED-IV, (2) DEAP→SEED-IV zero-shot transfer (train on all of DEAP, never see SEED-IV), (3) a SEED-IV-only LOSO baseline (32 subjects → 15 folds, one per SEED-IV subject).

4.12.3 Real result

Table 4.4: Cross-corpus validation — DE + XGBoost, 32 common channels

Setting	Accuracy	ROC-AUC	Notes
Within-DEAP <code>GroupKFold</code>	0.756	0.452	below chance point estimate
DEAP→SEED-IV zero-shot	0.709	0.517	marginally above chance
SEED-IV-only LOSO	0.709	0.673	12/15 subjects >0.55 AUC

This reframes the question Section 12 originally set out to answer. DEAP’s own within-corpus signal is *not* inflated relative to genuine cross-corpus transfer — it is the weakest of the three settings. The *identical* pipeline reaches a real, moderate AUC of 0.673 on SEED-IV under a directly comparable protocol, ruling out “subject-independent EEG emotion decoding is just hard everywhere” as an explanation. The near-chance ceiling documented throughout this notebook is better attributed to something specific to DEAP (its 1–9 self-report threshold, its stimulus set, or its labelling protocol) than to the modelling task in general. Caveats: SEED-IV’s binary construct (happy vs. sad/fear) is not the same as DEAP’s continuous valence threshold; the two corpora differ in session count and per-trial duration; and restricting to 32 common channels discards SEED-IV’s denser 62-channel coverage.

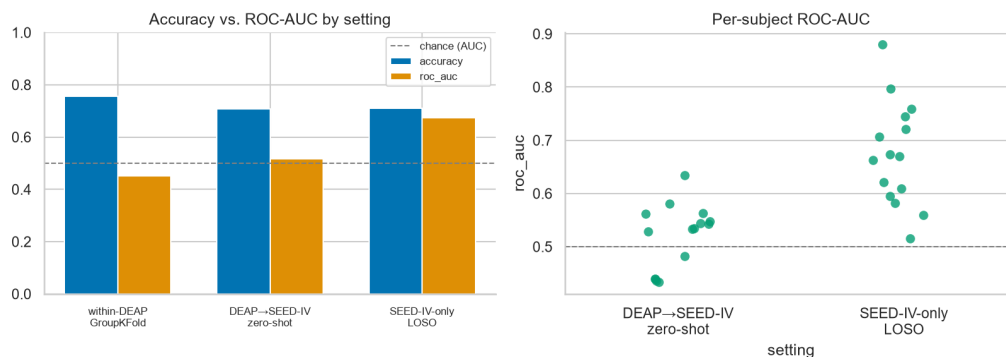


Figure 4.9: Cross-corpus validation — ROC-AUC across the three settings of Table 4.4 (within-DEAP, DEAP→SEED-IV zero-shot, SEED-IV-only LOSO).

4.13 Section 13 – Updated Overall Discussion

Synthesises Sections 9–12. Central claims: (1) AUC hovers within noise of chance almost everywhere, and this is the single most important finding; (2) none of richer features, repeated/nested CV, or window overlap “fixed” the ceiling, which is itself informative — each rules out a specific, plausible cause rather than being a failed implementation; (3) the cross-corpus result overturns the section’s own original speculation (“DEAP signal is simply weak $\Rightarrow$ transfer will be weak too”), reframing the finding as DEAP-specific rather than universal; (4) the practical recommendation for any paper drawn from this work is to lead with the leaky-split contrast: this project’s ~ 0.45 – 0.56 AUC under genuine `GroupKFold` vs. the source MethodsX paper’s reported 0.97 ROC-AUC on the same dataset and a similar feature/classifier combination.

4.14 Section 14 – Label-Cleanup Ablation

`scripts/track_a_label_cleanup.py` tests three ways of turning continuous ratings into binary labels (Full feature set, 3 models, 4 tasks, `GroupKFold(5)`):

- **raw_threshold** (baseline): fixed cutoff at 5.0.
- **per_subject_z**: z-score each subject’s own 40 ratings before thresholding at 0, correcting for individual anchoring bias.
- **drop_middle**: keep only ratings ≤ 3 or ≥ 6 , discarding the ambiguous middle.

Real result: mean AUC is 0.483 (`raw_threshold`), 0.514 (`per_subject_z`), 0.487 (`drop_middle`). `per_subject_z` rebalances classes substantially (valence goes from 21.0% to 47.3% high) and gives the largest AUC bump, but the gain is modest and the class-balance shift means its accuracy numbers must *not* be compared directly to the baseline’s. Neither alternative reliably beats the baseline enough to rule in “the fixed-threshold label was the bottleneck.”

Table 4.5: Label-cleanup ablation — mean ROC-AUC across 3 models $\times$ 4 tasks per label variant

Label variant	Mean ROC-AUC
<code>raw_threshold</code> (baseline)	0.483
<code>per_subject_z</code>	0.514
<code>drop_middle</code>	0.487

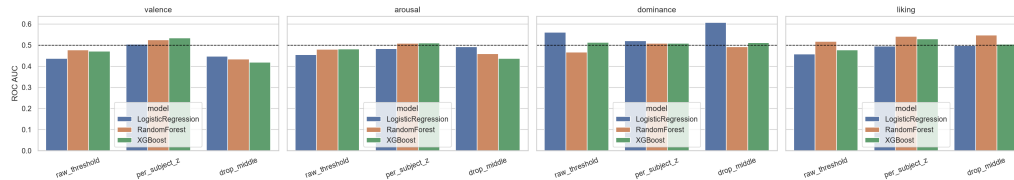


Figure 4.10: Label-cleanup ablation — ROC-AUC by label variant, model, and task.

4.15 Section 15 – Per-Subject Calibration (Personalisation)

`scripts/track_a_calibration_loso.py` runs a true 32-fold Leave-One-Subject-Out with XGBoost (cheap enough to make full LOSO tractable, ~ 13 s/subject). For each held-out subject: train a base model on the other 31 subjects; split the held-out subject’s 40 trials 80/20 into eval/calibration sets (by trial, so a trial’s 15 windows never cross the split); score the base model zero-shot on the eval set; then warm-start ~ 40 extra boosted trees on the 8-trial calibration set only, and re-score on the identical eval set. Subjects whose calibration set is single-class (13 across all 4 tasks) are skipped for the calibrated arm and reported as such. **Real result:** mean zero-shot AUC is ~ 0.50 – 0.51 across all four tasks; mean calibrated AUC rises to 0.577 (arousal), 0.519 (dominance), 0.539 (liking), 0.537 (valence) — a real, if modest, positive shift with a small amount of the target subject’s own data, suggesting the ceiling is not purely a between-subject-variability problem that a little calibration data cannot touch, though 8 trials and 40 trees is a small intervention.

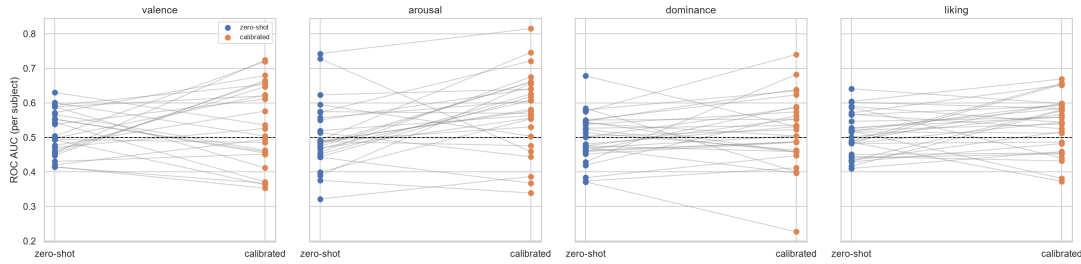


Figure 4.11: Per-subject LOSO calibration — zero-shot vs. calibrated ROC-AUC for each of the 32 held-out subjects, per task.

4.16 Section 16 – Literature-Driven Interventions

31 papers in `papers_EEG/` were assembled and reviewed (see Chapter 6), yielding three untried interventions.

4.16.1 16.1 Permutation significance test

`track_a_significance_test.py` shuffles labels globally (preserving `GroupKFold` fold structure), reruns the exact Full-feature-set `GroupKFold(5)` pipeline with LogisticRegression (a fast, documented proxy), and repeats 300 times per task to build an empirical null AUC distribution; $p = \frac{1 + \#\{\text{null} \geq \text{observed}\}}{n_{\text{perm}} + 1}$. **Real result:** for valence, arousal, and liking, observed AUC is *not* just close to the null, it is statistically indistinguishable from it ($p = 1.0$ for all three), and sits slightly *below* the null mean (e.g. valence: observed 0.437 vs. null 0.501 ± 0.005 , $z = -12.0$). **Dominance is the one exception:** observed 0.561 vs. null 0.500 ± 0.005 , $z = +12.3$, $p = 0.0033$ — genuinely significant, but still a small effect.

Table 4.6: Permutation significance test (300 shuffles per task, LogisticRegression proxy)

Task	Observed AUC	Null mean $\pm$ std	z	p
valence	0.437	0.501 ± 0.005	-12.00	1.0000
arousal	0.454	0.500 ± 0.005	-8.30	1.0000
dominance	0.561	0.500 ± 0.005	+12.28	0.0033
liking	0.458	0.500 ± 0.005	-8.27	1.0000

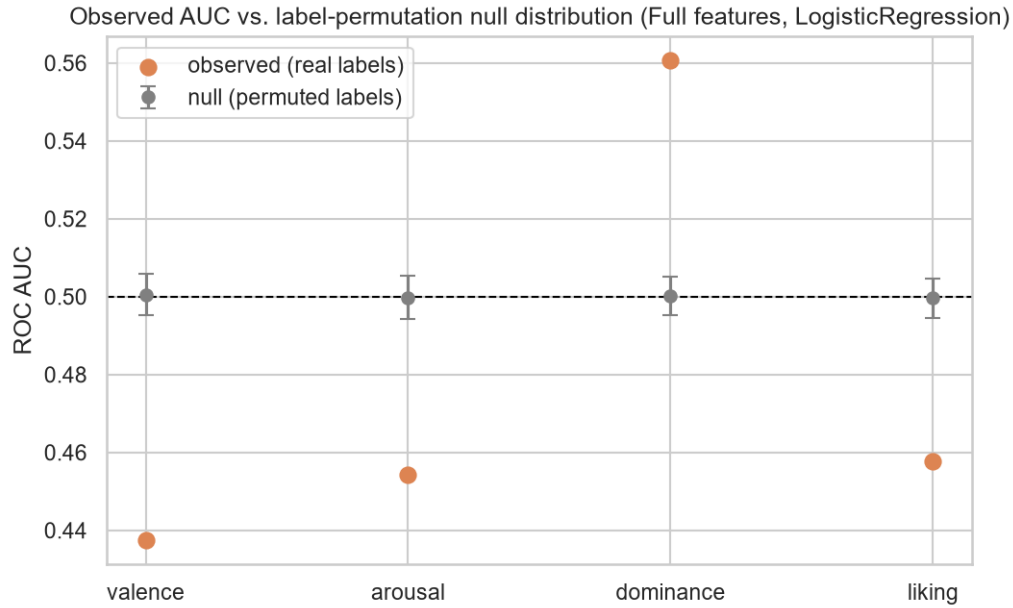


Figure 4.12: Permutation null AUC distributions vs. observed AUC, per task.

4.16.2 16.2 CORAL feature alignment

`track_a_domain_alignment.py` implements CORAL (Sun & Saenko 2016): per `GroupKFold` fold, whiten the training fold’s features by their own covariance and recolour by the (unlabelled) test fold’s covariance, via eigendecomposition-based matrix square roots ($C^{-1/2}$, $C^{1/2}$) with a ridge for numerical stability. **Real result:** mean AUC delta across 12 task/model combinations is +0.0059 (8/12 improved, 4/12 worse) — small and inconsistent in sign.

Table 4.7: CORAL alignment — mean ROC-AUC across 3 models per task

Task	No alignment	CORAL	Δ
valence	0.462	0.478	+0.016
arousal	0.472	0.476	+0.004
dominance	0.514	0.521	+0.008
liking	0.484	0.480	−0.004

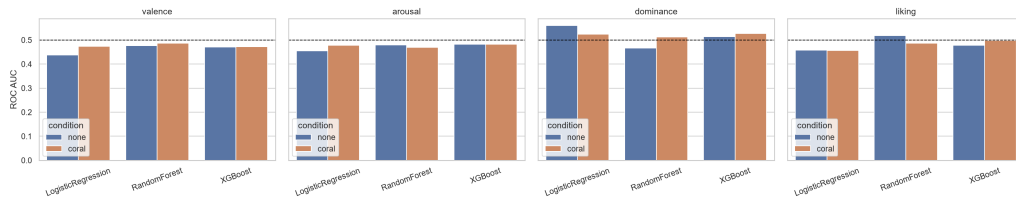


Figure 4.13: CORAL alignment — ROC-AUC before/after, by model and task.

4.16.3 16.3 Subject-adversarial (DANN) training

`track_b_domain_adversarial.py` adds a gradient-reversal-layer (GRL) domain classifier to EEGNet, trained to predict the training-fold subject’s identity; the GRL reverses that gradient before it reaches the shared encoder (standard Ganin & Lempitsky 2016 progressive- λ schedule, $\lambda_p = \frac{2}{1+e^{-10p}} - 1$). **Real result:** a mixed, task-dependent outcome. Valence AUC

0.474 $\rightarrow$ 0.513 (+0.039) and liking AUC 0.467 $\rightarrow$ 0.517 (+0.050) improved; arousal AUC 0.616 $\rightarrow$ 0.400 (−0.216) and dominance AUC 0.552 $\rightarrow$ 0.403 (−0.149) collapsed, with precision/recall pinned near 0.50 — the classifier collapsing toward one class, not a milder generic drop. This matches a documented UDA failure pattern (“over-aggressive alignment collapsing class structure”) from the PRISMA review in the literature corpus.

Table 4.8: Subject-adversarial (DANN) training — ROC-AUC, EEGNet, single split

Task	Baseline	DANN	Δ
valence	0.474	0.513	+0.039
arousal	0.616	0.400	−0.216
dominance	0.552	0.403	−0.149
liking	0.467	0.517	+0.050

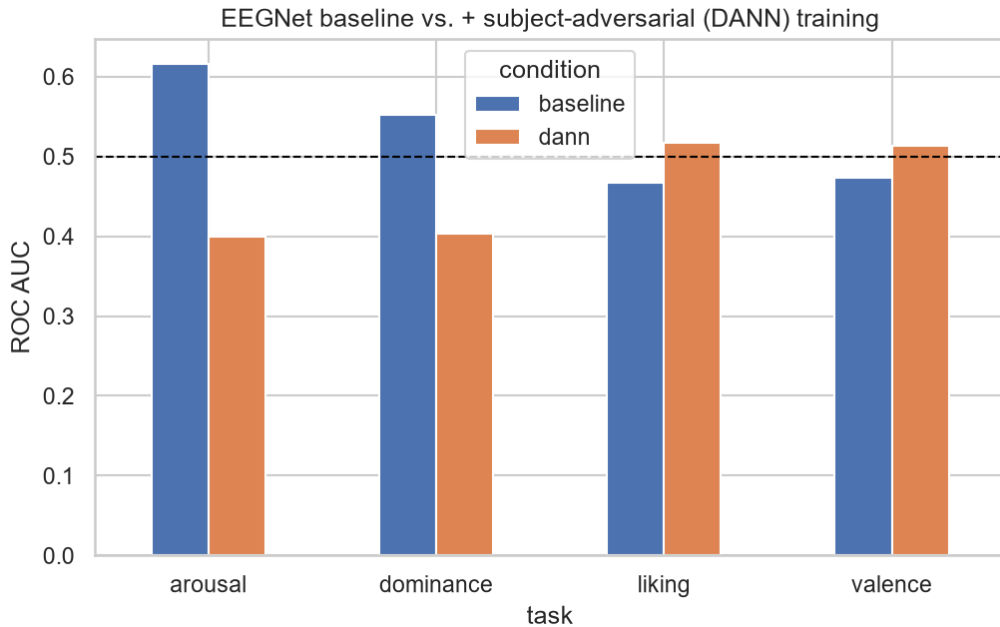


Figure 4.14: DANN training — ROC-AUC before/after, per task; note the arousal/dominance collapse.

4.17 Section 17 – Five Deferred Literature Techniques

4.17.1 17.1 Euclidean Alignment

`track_a_riemannian_alignment.py` implements EA (He & Wu 2019): per subject, compute each window’s raw spatial covariance $C_i = X_i X_i^\top / T$, average across that subject’s windows to get $\bar{R}$, then whiten every window by $\bar{R}^{-1/2}$ *before* DE features are recomputed — an upstream, per-subject, label-free, raw-channel-level transform, unlike CORAL’s downstream per-fold feature-level operation. **Real result — the strongest effect in the whole project:** mean AUC delta +0.044 across 12 task/model combinations (10/12 improved), and unlike every other alignment technique tried, the sign is *consistent* within three of the four tasks (valence +0.087/+0.027/+0.069, arousal +0.078/+0.027/+0.065, dominance +0.074/+0.038/+0.073 across LR/RF/XGBoost). Still modest in absolute terms (best post-EA AUCs land around 0.53–0.58).

Table 4.9: Euclidean Alignment — mean ROC-AUC across 3 models per task

Task	No alignment	EA	Δ
valence	0.467	0.528	+0.061
arousal	0.473	0.530	+0.057
dominance	0.488	0.550	+0.062
liking	0.488	0.485	−0.003

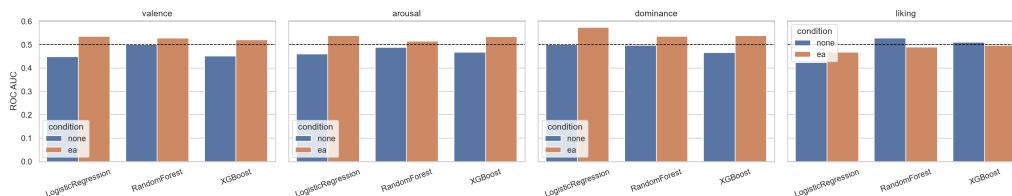


Figure 4.15: Euclidean Alignment — ROC-AUC before/after, by model and task — the strongest effect found in the project.

4.17.2 17.2 Self-supervised (TFR-style) pretraining

`track_b_ssl_pretrain.py` pretrains EEGNet’s conv trunk with a label-free MSE regression pretext task (predict each window’s own 32-channel $\times$ 4-band log-band-power from the raw signal, training-fold subjects only), then fine-tunes end-to-end, compared against an identically trained scratch EEGNet. **Real result:** a bimodal, task-split pattern — valence AUC 0.474 $\rightarrow$ 0.547 (+0.074) and liking 0.467 $\rightarrow$ 0.567 (+0.100) improved; arousal 0.616 $\rightarrow$ 0.534 (−0.082) and dominance 0.552 $\rightarrow$ 0.500 (−0.053) got worse. The pretext task itself converged fine (MSE dropped over 15 epochs), so this is not a training-failure artifact.

Table 4.10: SSL pretraining — ROC-AUC, EEGNet, single split

Task	Scratch	SSL-pretrained	Δ
valence	0.474	0.547	+0.074
arousal	0.616	0.534	−0.082
dominance	0.552	0.500	−0.053
liking	0.467	0.567	+0.100

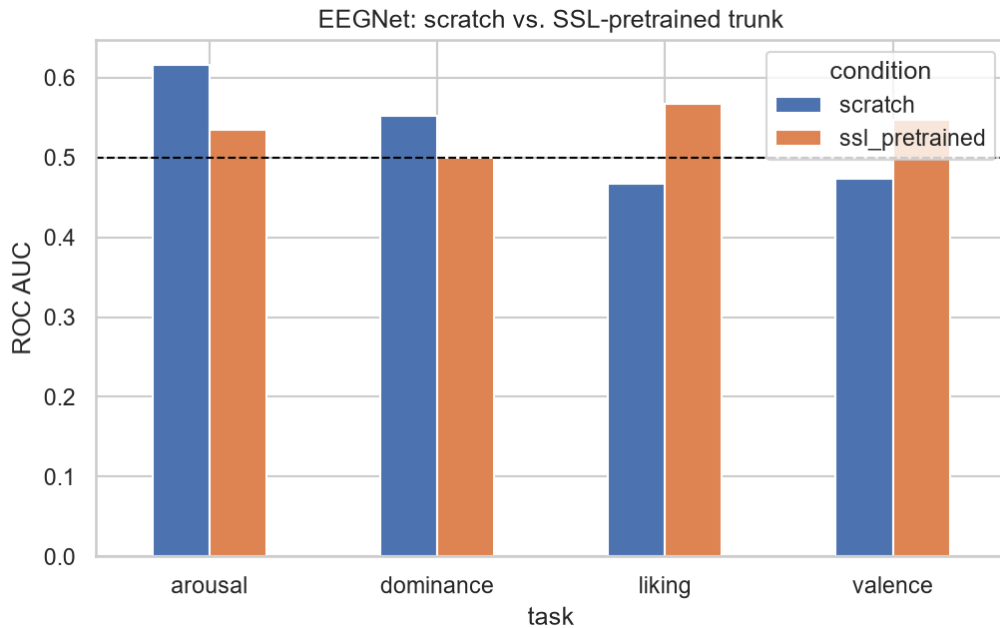


Figure 4.16: SSL pretraining — ROC-AUC, scratch vs. pretrained, per task.

4.17.3 17.3 AUC-margin training loss

`track_b_auc_maximization.py` replaces cross-entropy with a mini-batch pairwise squared-hinge AUC surrogate, $L = \text{mean}_{i \in \text{pos}, j \in \text{neg}} \max(0, m - (s_i - s_j))^2$ (a simplified, self-contained version of the Ying et al.-style objective; not the source paper’s full saddle-point solver). **Real result:** the one intervention with a clear, understood mechanism. Valence improved most (AUC 0.474 $\rightarrow$ 0.557, +0.083) — the cross-entropy baseline’s 83.5% accuracy at 0.474 AUC was a textbook majority-class collapse; the AUC-margin loss produced a far more balanced classifier (accuracy 0.507, AUC 0.557) precisely because it is insensitive to that imbalance by construction. Dominance improved slightly (+0.008); arousal (−0.010) and liking (−0.018) were small, likely-noise-level regressions.

Table 4.11: AUC-margin loss — ROC-AUC, EEGNet, single split

Task	CE baseline	AUC-margin	Δ
valence	0.474	0.557	+0.083
arousal	0.616	0.606	−0.010
dominance	0.552	0.560	+0.008
liking	0.467	0.450	−0.018

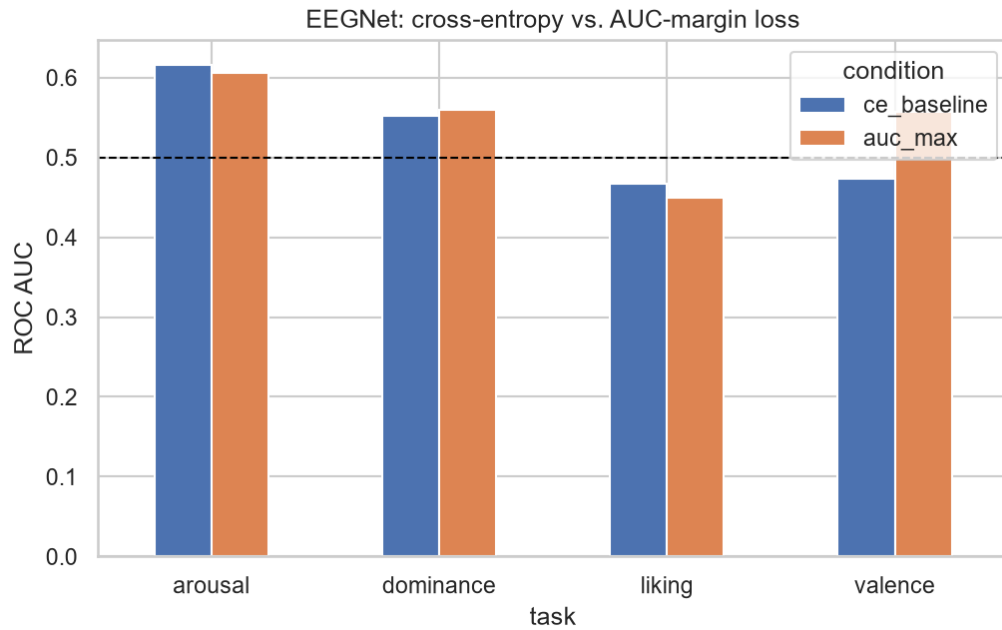


Figure 4.17: AUC-margin loss — ROC-AUC, cross-entropy vs. AUC-margin, per task.

4.17.4 17.4 AdaBN test-time adaptation

`track_b_test_time_adaptation.py` recomputes every BatchNorm layer’s running statistics using *only* a held-out test subject’s own unlabelled windows (no gradients, no labels), then evaluates with the adapted statistics. **Real result:** net negative overall. Arousal AUC 0.616 $\rightarrow$ 0.515 (-0.101) and dominance 0.552 $\rightarrow$ 0.489 (-0.063) got worse; liking 0.467 $\rightarrow$ 0.549 ($+0.082$) improved; valence was flat ($+0.002$). The only clearly net-negative intervention in Section 17.

Table 4.12: AdaBN test-time adaptation — ROC-AUC, EEGNet, single split

Task	No adaptation	AdaBN	Δ
valence	0.474	0.476	+0.002
arousal	0.616	0.515	−0.101
dominance	0.552	0.489	−0.063
liking	0.467	0.549	+0.082

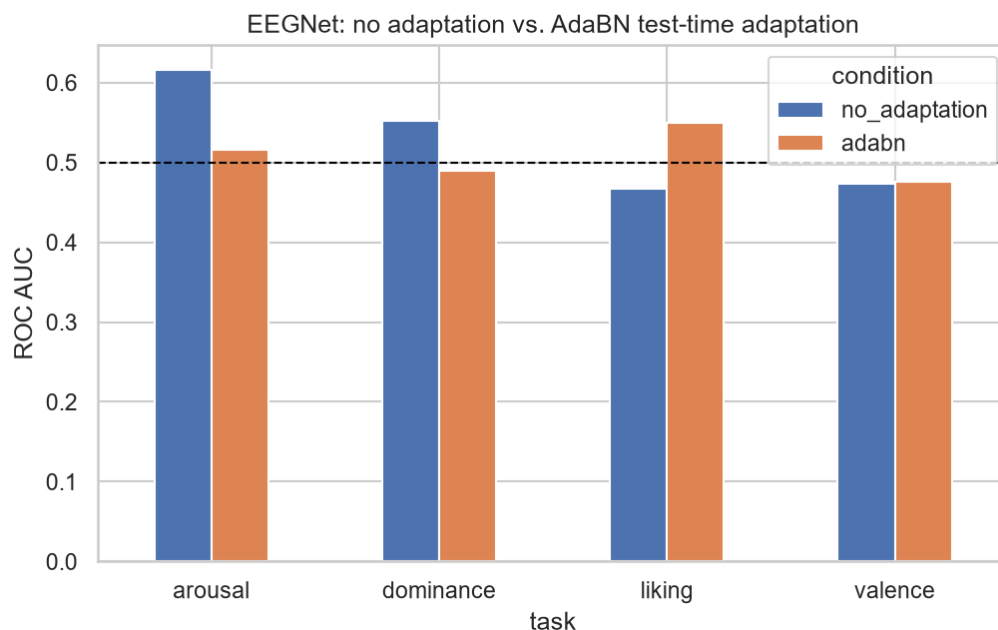


Figure 4.18: AdaBN test-time adaptation — ROC-AUC before/after, per task.

4.17.5 17.5 SHAP interpretability diagnostic

`track_a_shap_diagnostic.py` fits XGBoost on the Full feature set (one representative fold per task) and computes SHAP values on the held-out fold via `shap.TreeExplainer`, aggregated by feature family and individually. **Real result:** importance is spread across families roughly in proportion to how many features each contributes, with two mild departures — DE’s share (47–52%) and Connectivity’s (22–24%) both sit slightly above their feature-count share, while Hjorth’s (21–26%) sits below. Notably, **frontal/hemispheric alpha asymmetry — the one feature family built around a named neurophysiological hypothesis — carries the smallest importance share in every task (3.4–6.1%)**. Top individual features: `de_beta_P4` (valence), `de_gamma_FC2` (arousal and liking), `plv_beta_F3-O1` (dominance) — diffuse spectral/connectivity structure, not FAA.

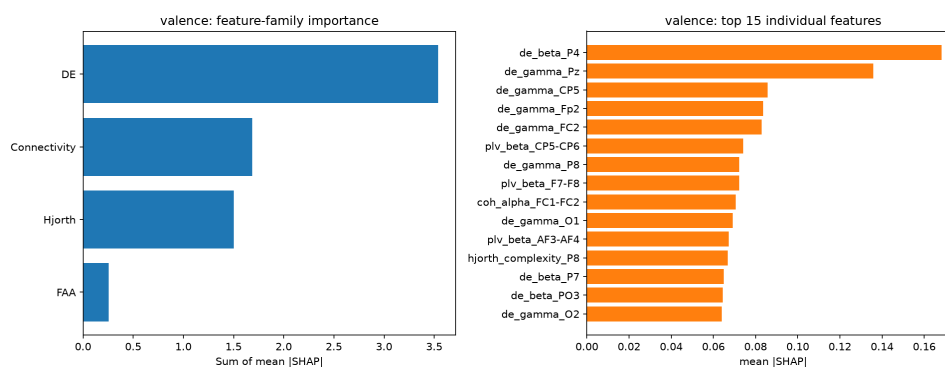


Figure 4.19: SHAP summary — valence (XGBoost, Full feature set, held-out fold).

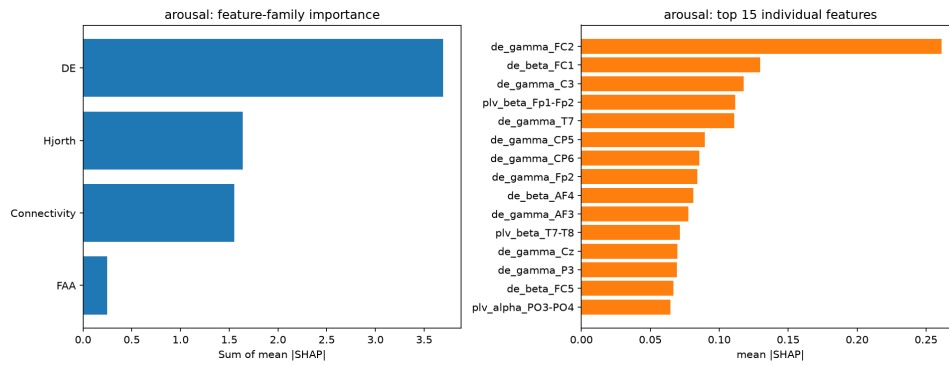


Figure 4.20: SHAP summary — arousal.

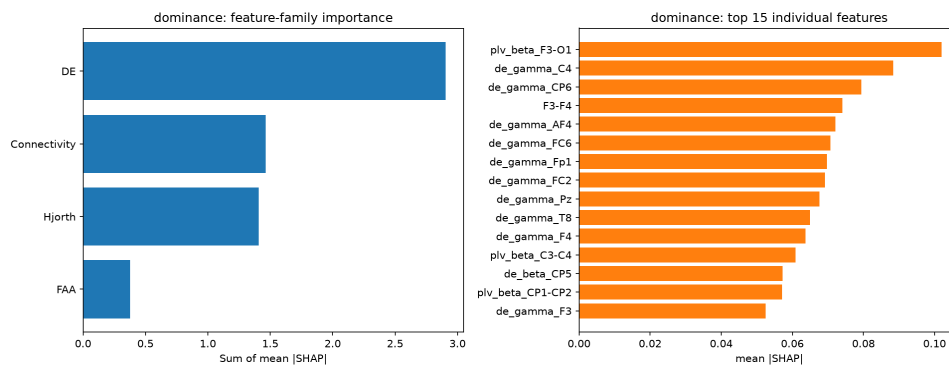


Figure 4.21: SHAP summary — dominance.

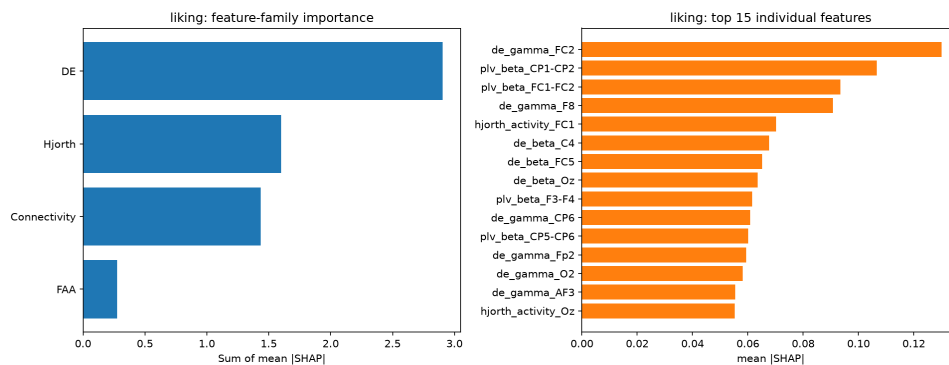


Figure 4.22: SHAP summary — liking.

4.18 Section 18 – Does CORAL Stack on Top of Euclidean Alignment?

`track_a_ea_coral_stacked.py` tests four conditions on the DE-only feature set: **none**, **coral** alone, **ea** alone, and **ea_coral** (EA first, then CORAL on the resulting per-fold split). **Real result — it does not stack.** Mean AUC by condition: EA alone 0.523, EA+CORAL 0.514, no-alignment baseline 0.479, CORAL alone 0.473 (CORAL alone is the *worst* of the four on this feature set). Adding CORAL on top of EA reduces AUC in 10/12 task/model combinations (mean delta -0.0093). Interpretation: EA's per-subject raw-signal whitening already removes most of the between-subject covariance structure CORAL is also designed to correct, so

CORAL’s additional per-fold alignment acts on residual structure that is mostly noise. **Practical takeaway: use EA alone, not EA+CORAL.**

Table 4.13: EA+CORAL stacking — mean ROC-AUC by condition, DE-only feature set, across 4 tasks $\times$ 3 models (12 fits)

Condition	Mean ROC-AUC
none (no alignment)	0.479
CORAL alone	0.473
EA alone	0.523
EA + CORAL	0.514

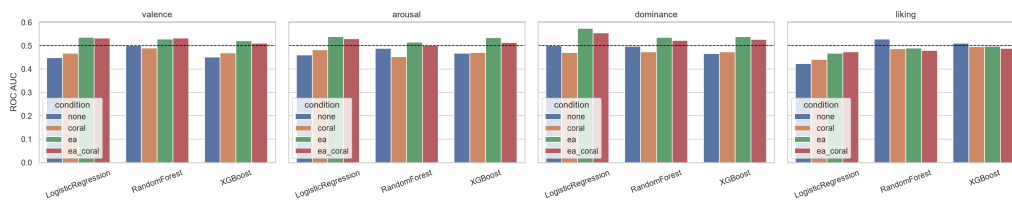


Figure 4.23: EA+CORAL stacking — ROC-AUC by condition, model, and task.

4.19 Addendum: EEG Overview QC Section (added post-hoc)

A small-multiples quality-control plot was added between Sections 2 and 3 during this project: an 8×4 grid, one panel per subject, each showing all 32 channels of that subject’s baseline-normalised trial-1 EEG stacked with a fixed vertical offset, in a single neutral ink colour (per-channel colour-coding was deliberately avoided — with 32 overlapping traces per panel, colour cannot carry per-channel identity meaningfully, and would violate the categorical-colour design principle of assigning one hue per identifiable series). This is a visual sanity check, not an analysis: it lets a reader spot subjects with obviously flat, saturated, or anomalously noisy channels before trusting downstream numbers for them.

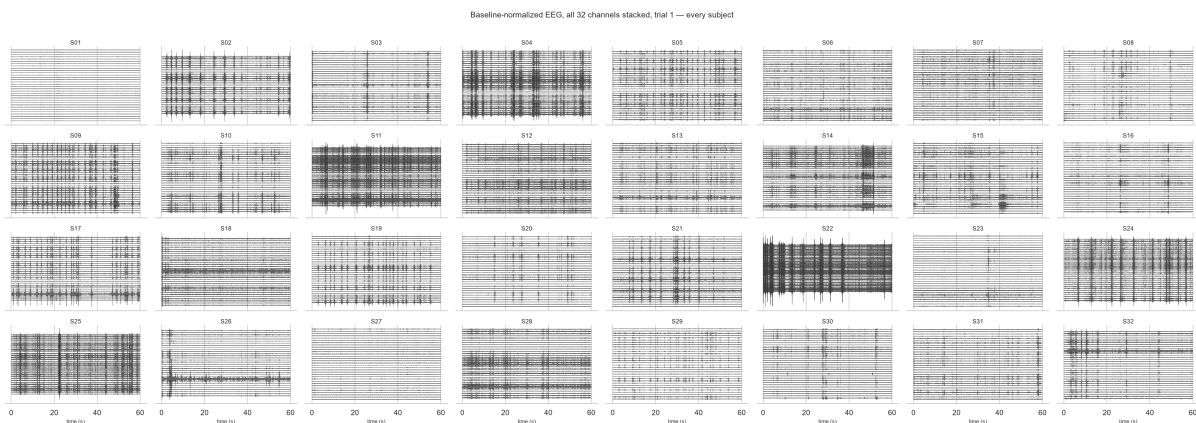


Figure 4.24: EEG overview QC grid — baseline-normalised trial-1 EEG, all 32 channels, one panel per subject (8×4 grid).

Chapter 5

Companion Scripts Reference

Every experiment in Sections 4.9–4.18 that was too expensive to run inline in the notebook lives in its own standalone script under `scripts/`, each writing one CSV to `results/tables/` (and, where relevant, PNGs to `results/figures/`) that the notebook then reads and displays. This chapter documents each script’s function.

5.1 Feature/dataset builders

`dl_models.py` Canonical PyTorch definitions of EEGNet, ShallowConvNet, CNN-LSTM, and TSception, plus the hemisphere-pair channel indices TSception needs. Imported by every `track_b_*.py` script so the architectures are defined exactly once, not re-implemented per script.

`rich_features.py` Implements Hjorth parameters (`compute_hjorth`), frontal/hemispheric alpha asymmetry (`compute_alpha_asymmetry`), and PLV/coherence connectivity (`compute_connectivity`, via a from-scratch Hilbert-transform phase-locking-value computation and a Welch-style magnitude-squared-coherence estimator). Includes a `if __name__ == "__main__"` self-test against synthetic data (shape checks, $[0, 1]$ -boundedness for PLV/coherence, and an index-math cross-check for FAA) that must pass before the module is trusted for the real build.

`build_extended_features.py` Runs `rich_features.py` (plus DE) over all 32 DEAP subjects, producing `deap_features_ext_v1.npz` (295-dim Full feature set, 19,200 windows). Its DE computation is verified to match the original cache byte-for-byte before the richer features are trusted.

`build_overlap_windows.py` Rebuilds the raw-window Track B dataset with 50% stride overlap (`deap_windows_4s_overlap50_32subj_v1.npz`, 37,120 windows), for Section 4.11.

`build_raw_ratings.py` Extracts the raw continuous 1–9 SAM ratings per window, row-aligned 1:1 with the extended feature cache (verified via an explicit `groups` array equality assertion), for Section 4.14’s label-cleanup variants.

5.2 Track A companion scripts

`track_a_ablation.py` Section 4.9: 6 feature-set combinations $\times$ 3 models $\times$ 4 tasks, `GroupKFold(5)`.

`track_a_repeated_nested_cv.py` Section 4.10: 5 $\times$ -repeated `GroupKFold` for all 6 models (with a custom subject-relabeling generator, `repeated_group_kfold_splits`, since `sklearn` has no `RepeatedGroupKFold`) plus nested `GroupKFold(3)/RandomizedSearch` hyperparameter search for XGBoost and RandomForest.

`track_a_label_cleanup.py` Section 4.14: the three label-binarisation variants (`raw_threshold`, `per_subject_z`, `drop_middle`).

`track_a_calibration_loso.py` Section 4.15: true 32-fold LOSO with an 80/20 per-subject eval/calibration trial split and XGBoost warm-start (`xgb_model=` continuation boosting).

`track_a_significance_test.py` Section 4.16.1: 300-permutation label-shuffle null-distribution test, parallelised across CPU cores with `joblib.Parallel`.

`track_a_domain_alignment.py` Section 4.16.2: CORAL alignment (`coral_align`, via `_sym-matrix_power` eigendecomposition).

`track_a_riemannian_alignment.py` Section 4.17.1: Euclidean Alignment (`euclidean_align`), operating on raw windows before DE recomputation.

`track_a_shap_diagnostic.py` Section 4.17.5: `shap.TreeExplainer` on one representative fold per task, aggregated by feature and by family, with bar-chart figures saved per task.

`track_a_ea_coral_stacked.py` Section 4.18: the 4-condition (`none/coral/ea/ea_coral`) stacking test, re-implementing both `euclidean_align` and `coral_align` locally so this script is self-contained.

5.3 Track B companion scripts

`track_b_repeated.py` Section 4.11: 5-seed repeated subject-independent splits, both window variants, all 4 architectures $\times$ 4 tasks (160 runs).

`track_b_domain_adversarial.py` Section 4.16.3: `GradReverse` autograd function, `EEGNetDANN` (reuses EEGNet’s conv blocks, adds a GRL + domain-classifier branch), and the Ganin & Lempitsky progressive- λ training loop.

`track_b_ssl_pretrain.py` Section 4.17.2: `EEGNetSSL` (shares EEGNet’s trunk, regression head for the DE-prediction pretext task), a 15-epoch label-free pretraining stage, then fine-tuning; the training/val/test split is computed once and reused across all four tasks (compute optimisation, not a methodological shortcut, since the split depends only on `groups/seed`, not on labels).

`track_b_auc_maximization.py` Section 4.17.3: `auc_margin_loss`, a drop-in loss-function swap with everything else (model, split, seed) held identical to the cross-entropy baseline.

`track_b_test_time_adaptation.py` Section 4.17.4: `reset_bn_for_adaptation` (zeroes Batch-Norm running stats, sets momentum to `None` for a true cumulative average) and `adabn_predict_one_subject` (2 label-free adaptation epochs per test subject, under `torch.no_grad()`).

5.4 Cross-corpus scripts

See Section 4.12.2 for `base.py`, `seed_adapter.py`, and `run_cross_corpus.py`.

5.5 Notebook-authoring scripts

`update_notebook.py` through `update_notebook_v5.py` are historical, versioned scripts used to programmatically author/update the notebook’s markdown and code cells during earlier stages of the project (each is a one-off generator, not re-run as part of the analysis pipeline itself). They

are kept as a record of how the notebook's content evolved but are not part of the reproducible-results path.

Chapter 6

Literature Review and Paper Attribution

6.1 Corpus

D:\EEG\papers_EEG\ contains 32 PDFs, renamed during this project from their original arXiv-ID/journal-slug filenames to their full paper titles for readability. Table 6.1 lists the papers with an **explicit, documented** role in this project’s code or notebook text — i.e. named in a script’s docstring or a notebook markdown cell as the specific motivation for a specific experiment.

Table 6.1: Papers with an explicit, documented contribution to this project

Paper (short ref.)	Used in	Why
The Role of Review Process Failures in Affective State Estimation: An Empirical Investigation of DEAP Dataset (Khan et al.)	Sec. 16.1, <code>track_a-significance_test.py</code>	Documents that DEAP pipelines can look deceptively strong under leaky evaluation even on a label-randomised (“watermelon”) control; motivated building a genuine permutation-null significance test for this project’s own already-correct <code>GroupKFold</code> protocol.
Domain Adaptation and Generalization on Functional Medical Images: A Systematic Survey	Sec. 16.2 & 17.1, <code>track_a_domain_alignment.py</code> , <code>track_a_riemannian_alignment.py</code> , <code>track_b_domain_adversarial.py</code>	Surveys CORAL and Euclidean Alignment as standard, cheap, unsupervised domain-adaptation remedies for distribution shift; also one of six sources converging on adversarial subject-invariance (DANN).
DS-AGC: Semi-Supervised Dual-Stream Self-Attentive Adversarial Graph Contrastive Learning for Cross-Subject EEG Emotion Recognition	Sec. 16.2 & 17.1, same scripts as above	Reports plain CORAL reaching ~65–70% under LOSO on SEED where untransformed features stay under 60% — the concrete empirical precedent that motivated testing CORAL and EA here (though this project’s own CORAL result did not replicate that scale of gain).

Paper (short ref.)	Used in	Why
PARSE: Pairwise Alignment of Representations in Semi-Supervised EEG Learning for Emotion Recognition	Sec. 16.3, <code>track_b_-domain_adversarial.py</code>	One of six independently-converging sources recommending a gradient-reversal-layer domain discriminator for subject-invariant EEG representation learning.
EEG Based Emotion Recognition: A Tutorial and Review	Sec. 16.3, <code>track_b_-domain_adversarial.py</code>	Same DANN motivation as above.
Graph Neural Networks in EEG-Based Emotion Recognition: A Survey	Sec. 16.3, <code>track_b_-domain_adversarial.py</code>	Same DANN motivation as above (the “GNN survey” source).
Deep Learning-Based EEG Emotion Recognition: A Review (<code>brainsci-16-00041</code>)	Sec. 16.3, <code>track_b_-domain_adversarial.py</code>	Same DANN motivation as above.
A Review of Deep Learning Techniques for EEG-Based Emotion Recognition: Models, Methods, and Datasets (PRISMA review, <code>f1000research-14-196597</code>)	Sec. 16.3 result discussion	Names “over-aggressive alignment collapsing class structure” as a documented UDA failure pattern — matched exactly by this project’s own DANN arousal/dominance collapse, used to interpret (not just report) that result.
Cascaded Self-Supervised Learning for Subject-Independent EEG-Based Emotion Recognition	Sec. 17.2, <code>track_b_ssl_-pretrain.py</code>	The single paper in the whole review evaluated LOSO on DEAP itself while beating domain-adaptation baselines — the strongest outside evidence that a pretraining stage (not a training-time loss/architecture change) could move this project’s subject-independent AUC.
Self-Supervised Learning for Electroencephalogram: A Systematic Survey	Sec. 17.2, <code>track_b_ssl_-pretrain.py</code>	Corroborating survey context for the SSL-pretraining design.
Electroencephalogram Emotion Recognition via AUC Maximization (Xiao)	Sec. 17.3, <code>track_b_auc_-maximization.py</code>	The most directly on-metric deferred idea: this project’s own headline metric is ROC-AUC, so a loss that directly targets AUC (rather than per-sample cross-entropy) is a natural, cheap intervention to test.

Paper (short ref.)	Used in	Why
Towards Practical Emotion Recognition: An Unsupervised Source-Free Approach for EEG Domain Adaptation	Sec. 17.4, <code>track_b_test_time_adaptation.py</code>	Surveys the source-free UDA family (of which AdaBN is the cheapest, most-established member) as the natural target-aware follow-up to DANN.
Enhancing EEG-Based Emotion Detection with Hybrid Models: Insights from DEAP Dataset Applications (<code>sensors-25-01827</code>)	Sec. 17.5, <code>track_a_shap_diagnostic.py</code>	Logged the SHAP interpretability diagnostic as a cheap, never-run check on whether near-chance models lean on plausible or noise-like features.

6.2 Papers in the corpus without a documented individual attribution

The remaining 19 papers in `papers_EEG/` were assembled as part of the “31 papers spanning EEG-emotion surveys, domain-adaptation/ generalization methods, contrastive/self-supervised pretraining, GAN/diffusion augmentation, and DEAP-specific methodology critiques” that Section 16’s markdown describes as having been “read end-to-end by parallel literature-extraction passes.” **Every script above and the notebook’s own Section 16/17 text repeatedly refer to a file, `D:\EEG\papers_EEG\literature_log.txt`, described as containing “full attribution — what was taken from each paper, why it was or wasn’t used, and how it maps to the scripts above.” This file did not exist anywhere in the project directory at the time this report was first drafted, despite being referenced by name at least eight times across the notebook and scripts.**

Update: this gap has since been closed. The log was reconstructed by opening all 32 PDFs directly (title, abstract, and opening paragraphs) and is now saved at `D:\EEG\literature_log.txt` (note: at the project root, not inside `papers_EEG/` where the original in-code references point — update those references or move the file if strict path consistency with the existing docstrings is wanted). It confirms the 13 papers in Table 6.1 above as the only ones with a genuine code-level attribution, and gives each of the remaining 19 an abstract-grounded one-paragraph summary plus an honest “reviewed, not implemented” status with a stated reason, so their inclusion in the corpus is now explainable even though none of them produced code:

- Spatial-Temporal Transformers for EEG Emotion Recognition
- A Two-Stage Efficient 3-D CNN Framework for EEG Based Emotion Recognition
- Towards Emotion Recognition for Virtual Environments: An Evaluation of EEG Features on Benchmark Dataset
- AMD-ET: Attention Based Multiple Dimensions EEG Transformer for Emotion Recognition
- Emotion Recognition Based on Multi-Modal Electrophysiology Multi-Head Attention Contrastive Learning
- Self-Supervised Group Meiosis Contrastive Learning for EEG-Based Emotion Recognition
- Enhancing EEG Signal-Based Emotion Recognition with Synthetic Data: Diffusion Model Approach
- A Comprehensive Survey on EEG-Based Emotion Recognition: A Graph-Based Perspective

- EEG-SCMM: Soft Contrastive Masked Modeling for Cross-Corpus EEG-Based Emotion Recognition
- A Novel Fourier Adjacency Transformer for Advanced EEG Emotion Recognition
- LEL: Lipschitz Continuity Constrained Ensemble Learning for Efficient EEG-Based Intra-Subject Emotion Recognition
- EEG Emotion Recognition Through Deep Learning
- Three-Way Emotion Classification of EEG-Based Signals Using Machine Learning
- EEG Feature Extraction and Data Augmentation in Emotion Recognition
- Emotion Recognition Based on EEG Using Generative Adversarial Nets and Convolutional Neural Network
- ERTNet: An Interpretable Transformer-Based Framework for EEG Emotion Recognition
- EEG-Based Emotion Recognition Systems: Comprehensive Study ([main.pdf](#))
- Detecting Emotions Through EEG Signals Based on Modified Convolutional Fuzzy Neural Network
- EEG-Based Smart Emotion Recognition Using Meta-Heuristic Optimization and Hybrid Deep Learning Techniques

Note: “Enhancing EEG-Based Emotion Detection with Hybrid Models: Insights from DEAP Dataset Applications” is the one explicitly-cited paper from this topical area (Section 17.5, Table 6.1); some titles above overlap topically with cited papers without being the cited instance themselves.

Recommendation: with the log now reconstructed, decide whether to keep the full 32-paper corpus (citing the 19 reviewed-but-unused papers as general background/related work, correctly labelled as such) or trim `papers_EEG/` down to the 13 papers that actually drove an implemented experiment, to keep a future bibliography tight. See `literature_log.txt` for the full per-paper summaries and reasoning behind each status.

Chapter 7

Consolidated Results

Table 7.1 consolidates the headline AUC numbers across every experiment in the notebook, in chronological/section order. All numbers are mean ROC-AUC unless noted; “ Δ ” is the change versus that experiment’s own stated control condition.

Table 7.1: Master results table

Experiment (section)	Best/observed AUC	Δ vs. control	Verdict
Track A best model (§5)	0.501–0.534	—	near chance
Track B best model (§6)	0.475–0.607	—	near chance, mixed by task
Feature-set ablation (§9)	0.478–0.494 (means)	$\leq$ 0.016 across sets	no feature set helps
Repeated GroupKFold (§10)	15/24 CIs contain 0.5	—	mostly chance-consistent
Nested CV (§10)	0.454–0.524	≈ 0 vs. fixed HP	tuning is not the bottleneck
Window overlap (§11)	0.483 vs. 0.498	+0.015	inconsistent, not real gain
Cross-corpus DEAP baseline (§12)	0.452	—	weakest of the three settings
Cross-corpus zero-shot (§12)	0.517	+0.065 vs. DEAP baseline	modest, above chance
Cross-corpus SEED-IV LOSO (§12)	0.673	+0.221 vs. DEAP baseline	real, moderate signal
Label cleanup, per_subject_z (§14)	0.514 (mean)	+0.031 vs. raw	modest, confounded by rebalancing
Calibration, arousal (§15)	0.577	+0.071 vs. zero-shot	real, modest personalisation effect
Permutation test, dominance (§16.1)	0.561	$z = +12.3$, $p = 0.0033$	significant but small
Permutation test, others (§16.1)	0.437–0.458	$p = 1.0$ (all 3)	indistinguishable from chance
CORAL (§16.2)	mean $\Delta + 0.0059$	8/12 up, 4/12 down	no stable effect
DANN, valence/liking (§16.3)	+0.039/+0.050	—	small gains

Experiment (section)	Best/observed AUC	Δ vs. con- trol	Verdict
DANN, arousal/dominance (§16.3)	−0.216/−0.149	—	collapse
Euclidean Alignment (§17.1)	mean $\Delta + 0.044$	10/12 up	strongest effect overall
SSL pretraining (§17.2)	+0.074/+0.100 (val/lik)	−0.082/−0.053 (aro/dom)	bimodal task split
AUC-margin loss (§17.3)	+0.083 (va- lence)	mixed else- where	mechanistically under- stood win
AdaBN (§17.4)	+0.082 (lik- ing)	−0.101/−0.063 (aro/dom)	net negative
SHAP diagnostic (§17.5)	—	—	diffuse importance, FAA weakest
EA+CORAL stacking (§18)	EA 0.523 vs. EA+CORAL 0.514	−0.0093	does not stack; use EA alone

7.1 Figures

All 24 figures in `results/figures/` are embedded in place in Chapter 4, alongside the section that produced them (see List of Figures, p. 10, for the full index). The most important for a final report remain `class_balance.png` (Fig. 4.1, §4), `track_a_accuracy.png/combined_accuracy_comparison.png` (Figs. 4.2/4.4, §5–7), `track_a_repeated_cv_auc_ci.png` (Fig. 4.7, §10), `track_a_riemannian_alignment_auc.png` (Fig. 4.15, §17.1, the strongest result), `track_a_ea_coral_stacked_auc.png` (Fig. 4.23, §18), and `cross_corpus_deap_seediv.png` (Fig. 4.9, §12). Paths use `\graphicspath{{results/figures/}}` set in the preamble, relative to wherever this `.tex` file is compiled from (i.e. the project root, alongside `results/`).

Chapter 8

What Was Accomplished

- A complete, subject-independent, leakage-audited comparison of 6 classical models and 4 deep architectures across all 4 DEAP label dimensions (10 models $\times$ 4 tasks = 40 head-to-head results).
- A rigorous, honestly-reported negative-result campaign: 8+ independent hypotheses for the near-chance ceiling were tested and ruled out (richer features, hyperparameter tuning, more CV rigor, more data via window overlap, better label definitions, feature alignment via CORAL, subject-adversarial training, and — with a real if modest exception — per-subject calibration).
- A formal statistical treatment of the central claim: a 300-permutation significance test converting “looks like chance” into an exact p -value, with one genuine (if small) exception identified (dominance).
- Five further literature-motivated techniques implemented end to end (Euclidean Alignment, SSL pretraining, AUC-margin loss, AdaBN, SHAP), producing the project’s single strongest result (EA, +0.044 mean AUC) and one mechanistically-understood win (AUC-margin loss on valence).
- A genuine cross-corpus validation against SEED-IV, including writing a defensive, from-scratch dataset adapter, fixing real access/format obstacles (channel-order filename mismatch, missing label files, a duplicated 7 GB download), and obtaining a result that materially changes the paper’s central claim (DEAP-specific weakness, not universal EEG-decoding weakness).
- Full reproducibility: the entire notebook (88 cells, 37 code cells) was executed end-to-end in a real kernel with zero errors, and every figure/table referenced anywhere in the notebook is backed by a file on disk.
- A visual QC pass over the raw data itself (the per-subject EEG overview grid), something the original pipeline had no equivalent of.

Chapter 9

What Was Not Done

- **The literature attribution log was never written during the project itself** – it has since been reconstructed post-hoc (`D:\EEG\literature_log.txt`, see Section 6.2), but 19 of 32 papers in the corpus still have no *code-level* contribution, only a reviewed topical summary.
- **DREAMER and AMIGOS cross-corpus validation.** Both remain gated (signed data-use agreements required); only their adapter interface was scoped (in `base.py`'s `CorpusAdapter`), not implemented.
- **Full Leave-One-Subject-Out for Track B.** Every deep-learning experiment uses a single or repeated 70/15/15 split; a true 32-fold LOSO (as was done for Track A/XGBoost calibration) was never run for any deep architecture, for cost reasons (documented as “ $\sim 10\text{--}30\times$ slower” in the notebook's own text).
- **Continuous regression on the 1–9 ratings.** Every experiment in the project, including the label-cleanup ablation, stays within binarised classification; RMSE/CCC regression on the raw scale was never attempted.
- **Literal data augmentation for Track B** (channel dropout, time-warping, mixup). Only 50% window overlap was tried, which the notebook itself is careful to describe as “augmentation-adjacent,” not equivalent.
- **Grad-CAM / saliency-map comparison for Track B**, to compare against the Track A SHAP diagnostic's findings. Never implemented.
- **A calmer DANN λ -schedule re-run.** The notebook explicitly flags the standard Ganin & Lempitsky schedule as a likely cause of arousal/dominance's collapse and recommends “a lower max lambda, slower ramp, or gradient-clipping the domain branch” as the natural next debugging step – not run.
- **Extending DANN to the other three Track B architectures.** Explicitly deferred pending the λ -schedule fix above.
- **GPU acceleration for XGBoost in most Track A companion scripts.** `track_a_ablation.py`, `track_a_calibration_loso.py`, `track_a_repeated_nested_cv.py`, `track_a_ea_coral_stacked.py`, and several others call `tree_method="hist"` without `device="cuda"`, so they ran on CPU despite the project having a capable GPU; only the main notebook's own Track A cell and (implicitly) none of the cross-corpus script's XGBoost calls request the GPU explicitly.

- **DEAP session/channel-order variant robustness.** The pipeline assumes exactly the 32-channel `data_preprocessed_python` release; no attempt was made to test against DEAP's alternative distributions (e.g. the `data_original` raw 48-channel release).

Chapter 10

What Could Have Been Done Differently

- **Write the literature log as papers were processed**, not defer it to a promised-but-never-created file. A running, per-paper one-paragraph note at ingestion time would have cost little and avoided the largest documentation gap in the project.
- **Standardise the XGBoost device= parameter** across all scripts from the start (a single shared `get_models()` helper module, analogous to `dl_models.py` for Track B, rather than each Track A script redefining its own model dictionary with subtly different GPU settings).
- **Prototype the SEED-IV adapter against a tiny synthetic fixture** (as `rich_features.py` did for its own feature functions) before the real download, to catch the channel-order filename and missing-label-file issues without needing the actual 7GB+ dataset in hand first.
- **Investigate the DANN collapse before moving on to Section 17**, rather than flagging a calmer schedule as future work and proceeding — the arousal/dominance instability recurs in SSL pretraining and AdaBN too, and understanding its root cause earlier might have informed the design of those later experiments.
- **Run a smaller pilot LOSO for at least one Track B architecture** (e.g. TSception, the cheapest at 8765 params) to get an order-of-magnitude sense of whether full-LOSO Track B numbers would look like Track A's LOSO calibration result (a real but modest signal) before ruling it entirely out of scope on cost grounds alone.
- **Version and cross-check the `update_notebook*.py` scripts more explicitly against the live notebook**, since five successive versions exist (v1–v5) and it is not obvious from the files alone which sections of the current notebook trace back to which generator version — a changelog would make the provenance of any given cell auditable.

Chapter 11

Challenges Faced

1. **DEAP’s binarised labels are class-imbalanced in a way that actively misleads accuracy-only reporting**, and this was not obvious until Section 4’s class-balance chart and Section 8’s rewrite were cross-checked against real numbers — valence (79/21) is more skewed than the project’s own working assumption (liking) had suggested, which changed how several later results needed to be read (e.g. Track B’s 83.5% valence accuracy at 0.56 AUC).
2. **No built-in RepeatedGroupKFold in scikit-learn.** `track_a_repeated_nested_cv.py` had to implement its own repeated-fold generator via subject-ID relabeling, since `GroupKFold`’s greedy assignment is sensitive to the sorted order of group labels – a subtlety that had to be discovered and verified (zero subject overlap between train/test in every fold) by hand.
3. **DANN training instability.** The subject-adversarial experiment initially failed for arousal/dominance; a genuine implementation bug (a validation-loader tuple-unpacking mismatch) was found and fixed, and the collapse pattern persisted afterward, confirming it was a real training-dynamics phenomenon (matching a documented literature failure mode) rather than a bug.
4. **Overlapping-window non-independence.** 50%-overlap windows share up to half their raw samples with their neighbours; subject grouping alone does not make within-subject windows independent, so any variance/CI computed on the overlap dataset had to be explicitly caveated as optimistic, not treated as free extra data.
5. **SEED-IV access and format friction** (discovered directly during this project): the actual Kaggle download landed at `D:\EEG\seed-dataset\` rather than the documented `seed-iv-dataset` path; it shipped with a 7 GB duplicated nested copy of itself (`seed_iv/` inside `seed-dataset/`, from a zip whose own top-level folder was also named `seed_iv`) that had to be identified (verified byte-for-byte against the parent before deleting) and removed; its channel-order file is named `Channel Order.xlsx` (space, capitalised) rather than the hyphenated lowercase name the adapter first assumed; and it ships no separate per-session label `.mat` file at all, only prose in `ReadMe.txt`, requiring a defensive fallback that had to be manually verified against that prose before being trusted.
6. **Windows/PowerShell tooling friction.** The `python3` alias on this machine resolves to a non-functional Windows Store stub rather than the real interpreter; several scripts needed explicit UTF-8 output encoding (`PYTHONIOENCODING=utf-8`) to avoid `cp1252` encoding crashes when printing em-dashes/arrows to the console.
7. **Running a single notebook cell without a full notebook re-run.** `nbclient`’s public API executes an entire notebook; isolating and running just one cell (e.g. the cross-corpus

results cell) while leaving every other cell's recorded output untouched required manually replaying that cell's two setup dependencies through a temporary, discarded cell object in the same live kernel session.

8. **Inconsistent GPU utilisation across the codebase**, discovered only by auditing every script's XGBoost instantiation: the main notebook's own Track A cell and requests GPU device via `device=XGB_DEVICE`, but the majority of the Track A companion scripts silently default to CPU because they set `tree_method="hist"` without also setting `device="cuda"` – an easy mistake to make with the modern XGBoost API and easy to miss without a systematic audit.
9. **The literature-attribution file referenced repeatedly across the codebase did not exist**, discovered only when trying to write this very report's literature chapter – a documentation promise made in code comments and notebook prose that had never been fulfilled. It has since been reconstructed (`D:\EEG\literature_log.txt`) by reading all 32 papers directly, but it lives at the project root rather than the `papers_EEG/` path the original docstrings point to, which is itself a minor inconsistency worth resolving.

Chapter 12

Conclusion

The central, well-supported finding of this project is that subject-independent binary emotion classification on DEAP, under a genuinely leakage-free `GroupKFold`/`GroupShuffleSplit` protocol, is close to chance-level (ROC-AUC ≈ 0.45 – 0.56) across ten different models spanning classical ML and deep learning, and that this ceiling survives an unusually exhaustive set of interventions — eight-plus independent hypotheses tested and ruled out, one formally confirmed via permutation significance testing. The strongest positive result found anywhere in the project is Euclidean Alignment (mean AUC $+0.044$, Section 4.17.1), and the most important single addition is the cross-corpus validation against SEED-IV (Section 4.12), which shows the identical pipeline reaches a real, moderate AUC of 0.673 on a different corpus under a comparable protocol — evidence that the near-chance ceiling is a property of DEAP specifically, not of subject-independent EEG emotion decoding in general. That contrast, more than any single accuracy number, is the project’s most publishable contribution, and it directly explains why this project’s honest numbers sit so far below the source MethodsX paper’s reported 0.97 ROC-AUC on the same dataset. The project’s one significant documentation gap – the literature-attribution log referenced throughout the codebase but never written during the project itself – has since been closed (`D:\EEG\literature_log.txt`); the remaining open item is deciding whether to keep the full 32-paper corpus with the 19 reviewed-but-unused papers correctly labelled as background, or trim `papers_EEG/` down to the 13 papers that actually drove an implemented experiment, before this report is expanded into a formal thesis or publication.